

RogueEngine: Plano para uma Ferramenta de Jogos Roguelike em C#

Bruno Rocha Guimarães¹

¹Curso de Ciência da Computação – Centro Universitário Jorge Amado (Unijorge)
Salvador – BA – Brasil

brg.brunorocha@gmail.com

Abstract. *This paper describes the plan to build **RogueEngine**, a tool focused on 2D roguelike games in C#/NET. Instead of using large general engines like Unity or Godot [Unity Technologies 2024; Godot Engine 2024], the project bets on a smaller scope: grid maps, characters, turn-based rules, random dungeons, limited vision, enemy movement, scripts, and exporting a playable game. Game rules stay separate from the screen (SadConsole) [SadConsole 2024; Gregory 2018; Martin 2017]. Work is planned in small steps, from a first playable version (v0.1) to release 1.0, with tests [xUnit.net 2024], data files in JSON [JSON Schema 2024], C# scripts [Microsoft .NET Team 2024], a simple editor [Avalonia Team 2024], and a Windows build [Microsoft Learn 2023]. Expected results follow roguelike practice [Berlin Interpretation 2008; Wild 2018; Dungeon Crawl Stone Soup Team 2023] and basic software planning [Sommerville 2015; Boehm 1988; Beck et al. 2001].*

Resumo. *Este trabalho apresenta o plano para criar a **RogueEngine**, uma ferramenta voltada a jogos roguelike em 2D — aqueles em que o mapa é uma grade, o jogador anda por turnos e, quando morre, geralmente começa de novo [Berlin Interpretation 2008]. Em vez de usar motores grandes como Unity ou Godot [Unity Technologies 2024; Godot Engine 2024], a ideia é focar só no que esse tipo de jogo precisa: mapa em quadradinhos, personagens, combate por turnos, masmorras geradas automaticamente, visão limitada, inimigos que perseguem o jogador, scripts e exportação do jogo pronto [Hendriks et al. 2013; Togelius et al. 2016]. A regra de ouro do projeto é simples: o SadConsole cuida da tela; o núcleo da engine cuida das regras [SadConsole 2024; Martin 2017; Gregory 2018]. O desenvolvimento foi pensado em etapas, da versão 0.1 (algo jogável) até a 1.0 (primeira versão pública), com testes [xUnit.net 2024], dados em JSON [JSON Schema 2024], scripts em C# [Microsoft .NET Team 2024], editor visual simples [Repenning et al. 2017; Avalonia Team 2024] e geração de executável para Windows [Microsoft Learn 2023]. Os resultados esperados se apoiam em jogos e estudos já publicados [Craddock 2015; Wild 2018; Dungeon Crawl Stone Soup Team 2023; Pressman and Maxim 2014; Sommerville 2015; Boehm 1988].*

Palavras-Chave: *Roguelike; motor de jogos; C#; geração procedural; planejamento de software.*

1. Introdução

Jogos *roguelike* costumam ter mapa em grade, combate por turnos, morte permanente e fases geradas de forma aleatória [Berlin Interpretation 2008; Adams and Dormans 2014]. São jogos diferentes dos que usam mundo contínuo e física em tempo real [Gregory 2018]. Ferramentas como Unity e Godot [Unity Technologies 2024; Godot Engine 2024] servem para quase qualquer jogo, mas isso traz muita configuração e código extra quando o objetivo é só um roguelike 2D [Lewis and McGuire 2014; Nystrom 2014]. Estudos sobre geração automática de mapas [Hendrikx et al. 2013; Togelius et al. 2011; Shaker et al. 2016] mostram que uma ferramenta feita para o gênero pode já trazer, de fábrica, masmorra, visão do jogador e caminho dos inimigos — sem o desenvolvedor montar tudo do zero.

Por isso nasce o projeto **RogueEngine**: uma engine menor, feita para esse tipo de jogo, buscando facilitar a vida de quem desenvolve [Craddock 2015; Wild 2018; Dungeon Crawl Stone Soup Team 2023]. A linguagem escolhida foi C# com .NET [Microsoft Corporation 2024; Pinheiro and Oliveira 2020], pela organização do código e pela facilidade de gerar o executável final [Microsoft .NET Team 2024; Microsoft Learn 2023]. Para desenhar o jogo na tela, entra o SadConsole [SadConsole 2024; MonoGame Team 2024], que trabalha bem com caracteres e mapas em grade [Berlin Interpretation 2008; Browne and Maire 2010].

A ideia central é separar bem as partes: *SadConsole é a tela; RogueEngine.Engine é a regra do jogo* [Martin 2017; Fowler 2002]. Assim dá para testar movimento, combate e turnos sem abrir janela gráfica [Gregory 2018; xUnit.net 2024]. Este texto explica o embasamento (Seção 2), como o sistema foi planejado, com diagramas (Seção 3), o que se espera alcançar (Seção 4) e as conclusões (Seção 5).

2. Fundamentação Teórica

2.1. O que é um roguelike?

O gênero nasceu com o jogo *Rogue* e ficou conhecido por mapa em grade, combate por turnos, morte que zera o progresso e fases que mudam a cada partida [Berlin Interpretation 2008; Adams and Dormans 2014; Koster 2004]. Trabalhos sobre a evolução do gênero [Browne and Maire 2010; Wild 2018] mostram que esse formato em “quadrados” facilita colocar inimigos, calcular o que o jogador enxerga e balancear o jogo [Nystrom 2014; Buckland 2005; Tozour 2002]. Jogos como *Dungeon Crawl Stone Soup* [Dungeon Crawl Stone Soup Team 2023] provam que uma base bem feita para roguelike aguenta anos de melhorias sem precisar reinventar tudo a cada versão.

2.2. Mapas e inimigos gerados automaticamente

Em roguelikes, a masmorra costuma ser montada sozinha pelo computador — salas ligadas por corredores, por exemplo [Hendrikx et al. 2013; Togelius et al. 2011; Shaker et al. 2016; Togelius et al. 2016]. A RogueEngine deve permitir repetir o mesmo mapa quando o jogador usar a mesma “semente” (*seed*) [Togelius et al. 2011; Hendrikx et al. 2011]. Para os inimigos perseguirem o jogador, entram métodos clássicos como A* [Hart et al. 1968; Patel 2014] e mapas de distância [RogueBasin 2010; Patel 2014]. O campo de visão [Patel 2016] define o que está visível, o que já foi visitado e o que ainda é desconhecido — parte importante da tensão do jogo [Berlin Interpretation 2008; Wild 2018].

2.3. Como organizar o código do jogo

Livros sobre motores de jogo [Gregory 2018] mostram que vale a pena separar “regras” e “tela”. Na RogueEngine, cada coisa do jogo (personagem, item, parede) é uma *entidade* que ganha pedaços de informação — posição, vida, aparência — e sistemas cuidam do movimento, combate e turnos [Gamma et al. 1994; Nystrom 2014; Bjorge and Torgersen 2015; Martin 2017; Sommerville 2015]. Não é o modelo mais purista que existe, mas é simples de entender e de testar.

2.4. Ferramentas escolhidas

C# com .NET [Microsoft Corporation 2024; Pinheiro and Oliveira 2020] foi escolhido por ser direto de usar e por aceitar bem arquivos de dados em JSON [JSON Schema 2024]. Scripts do jogo podem ser escritos em C# e compilados junto com o projeto [Microsoft .NET Team 2024]. O SadConsole [SadConsole 2024] desenha o mapa na tela; por baixo dele está o MonoGame [MonoGame Team 2024], que cuida da janela e do teclado [Craddock 2015; SadConsole 2024]. O editor do projeto deve ser uma janela desktop em Avalonia [Avalonia Team 2024; Martin 2017; Fowler 2002]. No fim, o jogo sai como pasta ou instalador para Windows [FireGiant 2024; Microsoft Learn 2023; Microsoft Corporation 2024].

2.5. Como o trabalho será conduzido

O plano segue entregas pequenas e frequentes [Boehm 1988; Beck et al. 2001; Pressman and Maxim 2014], para não crescer demais e virar um projeto impossível [Lewis and McGuire 2014]. Cada etapa tem metas claras (CA-1 a CA-12) que dizem se aquela parte está pronta ou não [Sommerville 2015; Pressman and Maxim 2014]. Testes automáticos no núcleo [xUnit.net 2024] ajudam a garantir que as regras continuem certas mesmo depois de mudanças [Gregory 2018; Martin 2017].

3. Processo de Desenvolvimento

Esta seção mostra como a RogueEngine deve ser construída, da versão 0.0 (só planejamento) até a 1.0 (primeira versão utilizável), usando os diagramas do projeto.

3.1. Visão geral e partes do sistema

A Figura 1 resume o fluxo: quem cria o jogo usa o **Editor**, que mexe no arquivo do projeto (game.reproj), testa o jogo pelo **Runtime** e pede a **Ferramenta de build** para gerar o executável [Gregory 2018; FireGiant 2024; Microsoft Learn 2023]. O runtime lê as regras na **Engine** e manda o SadConsole desenhá-la na tela.

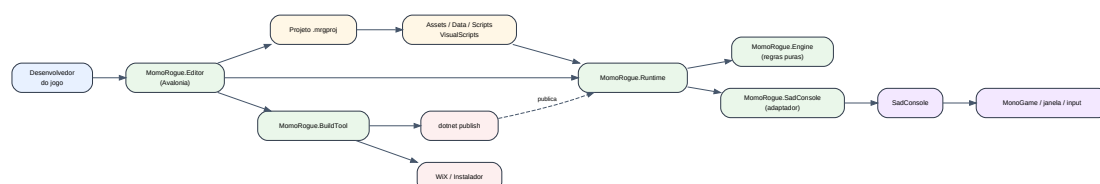


Figura 1. Visão geral: editor, projeto do jogo, runtime, engine, tela e exportação.

A Figura 2 mostra por dentro da engine: mundo do jogo, mapa, personagens, regras de movimento e combate, inimigos, geração de fases, arquivos de dados e scripts [Nystrom 2014; Buckland 2005; Hendrikx et al. 2013; Togelius et al. 2016; Repenning et al. 2017].

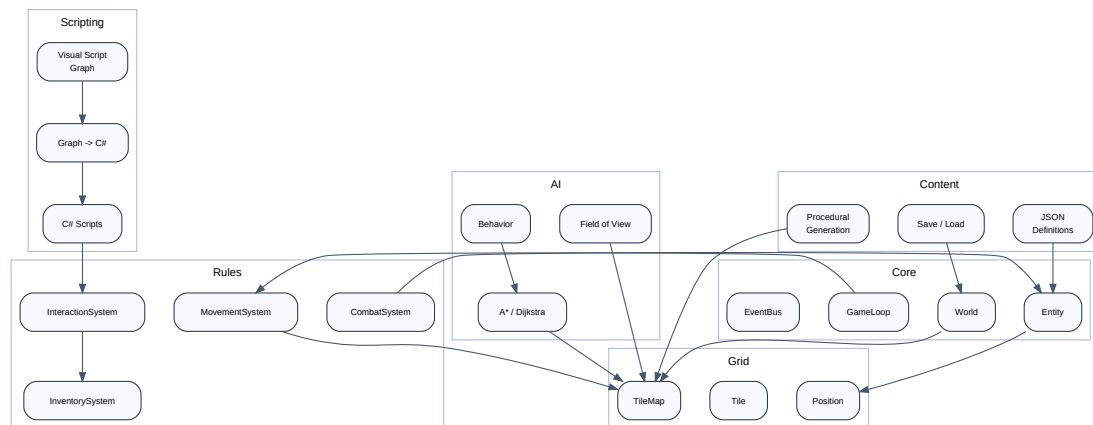


Figura 2. Partes internas da engine: núcleo, mapa, regras, inimigos, conteúdo e scripts.

O projeto divide-se em módulos (Engine, SadConsole, Runtime, Editor, BuildTool). A regra é: o núcleo não depende da tela; a tela depende do núcleo [Martin 2017; Gamma et al. 1994; Gregory 2018].

3.2. Peças principais do núcleo

A Figura 3 mostra as classes centrais: o *World* guarda o mapa e os personagens; cada *Entity* carrega componentes; sistemas atualizam movimento e combate; *IRenderer* é só a ponte para desenhar [Gregory 2018; Nystrom 2014; Bjorge and Torgersen 2015]. O *TurnManager* organiza a vez do jogador e dos inimigos [Berlin Interpretation 2008; Nystrom 2014; Buckland 2005].

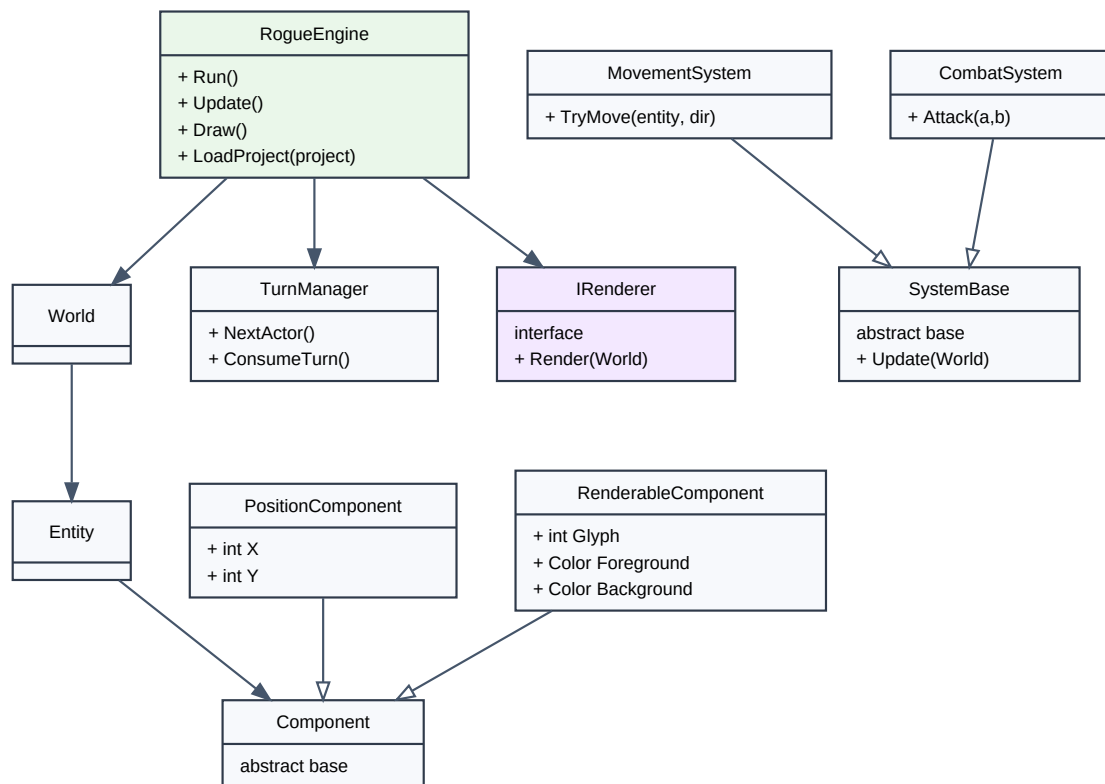


Figura 3. Classes principais: mundo, entidades, componentes, sistemas e renderização.

3.3. Etapas de implementação

O trabalho foi dividido em sete etapas [Boehm 1988; Sommerville 2015; Pressman and Maxim 2014]:

Etapa 1 (v0.1) — Primeiro protótipo jogável: montar o projeto .NET, mapa, personagem, teclado virando ação, movimento e colisão com paredes [Craddock 2015; SadConsole 2024; xUnit.net 2024]. Metas CA-2 e CA-3.

Etapa 2 (v0.2) — Roguelike básico: masmorra aleatória, turnos, inimigo que segue o jogador, combate simples e mensagens na tela [Hendrikx et al. 2013; Togelius et al. 2016; Buckland 2005; RogueBasin 2010; Berlin Interpretation 2008; Dungeon Crawl Stone Soup Team 2023]. Metas CA-4 a CA-6.

Etapa 3 (v0.3) — Projeto e dados: arquivo `game.reproj`, personagens e itens em JSON, salvar/carregar partida, modelo de projeto para copiar [JSON Schema 2024; Fowler 2002]. Metas CA-1, CA-7 e CA-10.

Etapa 4 (v0.4) — Scripts em C#: comportamentos escritos em código, compilados na hora de gerar o jogo, com mensagens de erro claras [Microsoft .NET Team 2024; Gamma et al. 1994]. Meta CA-8.

Etapa 5 (v0.5) — Gerar o jogo: comando `rogueengine build`, checagem dos arquivos, cópia de imagens e sons, pasta ou ZIP pronto para jogar [Microsoft Learn 2023; Pressman and Maxim 2014]. Metas CA-11 e CA-12.

Etapa 6 (v0.6) — Editor: programa para criar projeto, editar dados e testar o jogo sem sair do editor [Avalonia Team 2024; Martin 2017].

Etapa 7 (v0.7) — Scripts visuais: montar lógica ligando blocos (evento, condição, ação) que viram C# na exportação [Repenning et al. 2017; Microsoft .NET Team 2024]. Meta CA-9.

3.4. O que o usuário pode fazer e como o jogo é exportado

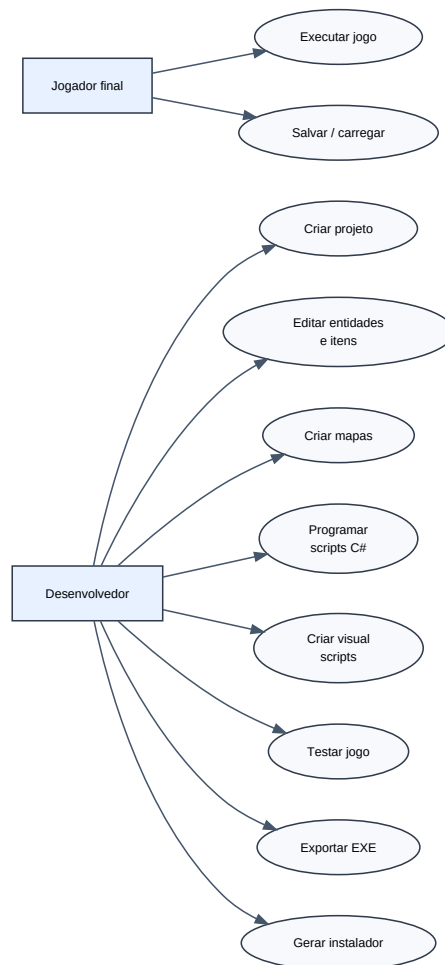


Figura 4. O que o criador do jogo e o jogador final podem fazer na plataforma.

A Figura 4 lista ações do desenvolvedor (criar projeto, editar personagens, mapas, scripts, testar, exportar) e do jogador (abrir o jogo, salvar e carregar) [Sommerville 2015; Adams and Dormans 2014; Pressman and Maxim 2014].

A Figura 5 mostra o caminho até o jogo pronto: validar arquivos, gerar código dos scripts visuais, compilar, copiar assets e publicar como ZIP, executável ou instalador [Microsoft Learn 2023; FireGiant 2024; Microsoft .NET Team 2024; Repenning et al. 2017].

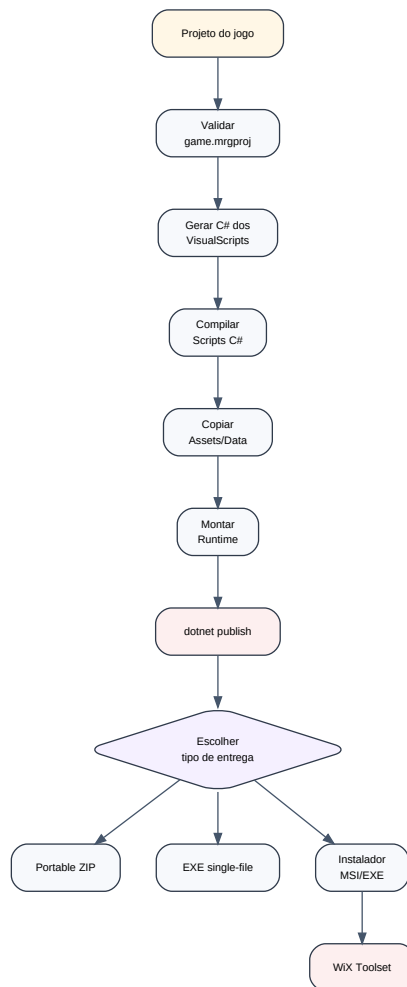


Figura 5. Passo a passo para exportar o jogo.

3.5. Turnos e estados do jogo

A Figura 6 explica um turno: o jogador aperta uma tecla, o jogo verifica se o movimento vale, passa a vez, os inimigos agem e a tela atualiza [Nystrom 2014; Buckland 2005; Berlin Interpretation 2008; Patel 2014].

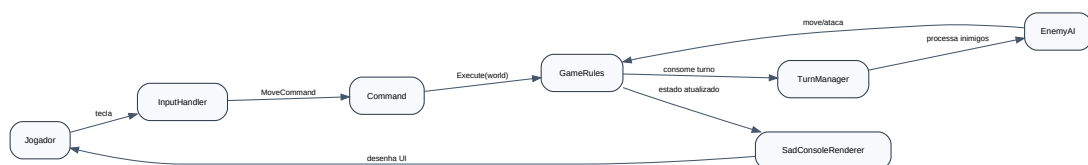


Figura 6. O que acontece em um turno de jogo.

A Figura 7 mostra os “modos” do programa: abrir projeto, menu, jogando, pausa, salvar, game over e sair [Nystrom 2014; Gregory 2018; Gamma et al. 1994].

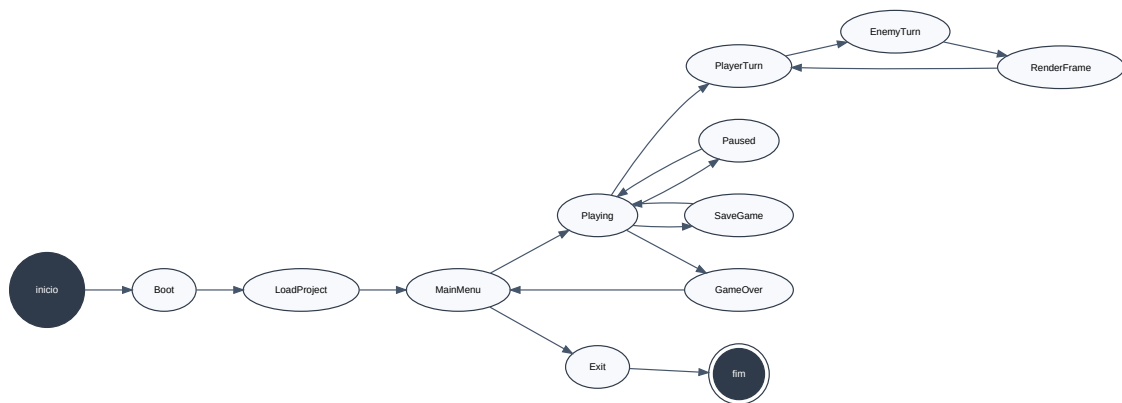


Figura 7. Estados do programa enquanto o jogo roda.

3.6. Scripts visuais

A Figura 8 mostra como um gráfico de blocos vira código C#, é compilado e roda no jogo; se der erro, o editor avisa [Repenning et al. 2017; Microsoft .NET Team 2024; Gamma et al. 1994].

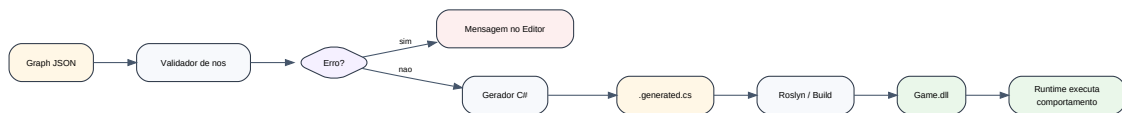


Figura 8. Do desenho dos blocos até o jogo executar o comportamento.

Na primeira versão, os scripts visuais cobrem situações simples (porta, spawn, mensagem), sem pretender substituir um editor gigante [Microsoft .NET Team 2024; Gregory 2018; Lewis and McGuire 2014; Repenning et al. 2017].

4. Resultados Esperados

Com base no que outros autores e projetos já mostraram [Gregory 2018; Hendrikx et al. 2013; Lewis and McGuire 2014], esta seção resume o que se espera alcançar.

4.1. No funcionamento do sistema

Separar regras e tela deve permitir testar o jogo “por dentro”, sem abrir janela, o que é mais rápido e confiável [Martin 2017; xUnit.net 2024; Gregory 2018]. Masmorras com salas e corredores, com opção de repetir o mesmo layout usando uma semente, seguem o que jogadores de roguelike já conhecem [Hendrikx et al. 2013; Togelius et al. 2011; Berlin Interpretation 2008; Dungeon Crawl Stone Soup Team 2023]. Inimigos que perseguem usando caminhos conhecidos [Hart et al. 1968; Patel 2014; RogueBasin 2010; Buckland 2005; Wild 2018] devem parecer com o comportamento de títulos clássicos. Scripts em C# compilados na exportação [Microsoft .NET Team 2024; Pinheiro and Oliveira 2020; Lewis and McGuire 2014] devem ser viáveis para projetos pequenos.

4.2. Na experiência de quem cria o jogo

O fluxo desejado é direto: criar projeto, editar arquivos de dados, testar e gerar o executável [Adams and Dormans 2014; Pressman and Maxim 2014; Sommerville 2015; Unity Technologies 2024; Godot Engine 2024; Lewis and McGuire 2014]. Tudo fica em pastas que podem ir para o Git [Fowler 2002; Beck et al. 2001]. O jogo final deve rodar no Windows sem instalação complicada [Microsoft Learn 2023; Craddock 2015; Pinheiro and Oliveira 2020].

4.3. Na forma de trabalhar

Cada versão (0.1 até 1.0) termina com metas objetivas (CA-1 a CA-12), para saber se o projeto está no trilho [Boehm 1988; Beck et al. 2001; Sommerville 2015; Pressman and Maxim 2014]. Ficou de fora, de propósito, coisas como multiplayer, loja de assets e física 3D — para não estourar o prazo [Lewis and McGuire 2014; Wild 2018]. Documentação e testes acompanham o código [Martin 2017; Sommerville 2015].

4.4. Limitações previstas

Scripts, na primeira versão, são código do próprio desenvolvedor — não há “caixa segura” para mods de desconhecidos [Gregory 2018; Microsoft .NET Team 2024; Pressman and Maxim 2014; Lewis and McGuire 2014]. O editor de blocos começa simples; não tenta competir com Unity ou Godot [Repenning et al. 2017; Unity Technologies 2024; Adams and Dormans 2014]. Versão para celular fica para depois [Pressman and Maxim 2014].

5. Conclusão

Este artigo apresentou o plano da **RogueEngine**, ferramenta para criar roguelikes 2D em C#/NET. A proposta parte de como esse tipo de jogo já é feito há anos [Berlin Interpretation 2008; Dungeon Crawl Stone Soup Team 2023; Wild 2018], de boas práticas de organização de software [Gregory 2018; Nystrom 2014; Martin 2017; Sommerville 2015; Boehm 1988; Pressman and Maxim 2014] e de um desenvolvimento em etapas pequenas [Beck et al. 2001]. Em vez de uma engine para tudo, o foco é mapa em grade, turnos, fases aleatórias, visão limitada, scripts e exportar o jogo [Hendrikx et al. 2013; Unity Technologies 2024; Godot Engine 2024; Lewis and McGuire 2014].

Os oito diagramas mostram como as peças se conectam e como o jogo sai do computador do desenvolvedor até virar um programa que qualquer pessoa pode abrir [FireGiant 2024; Microsoft Learn 2023]. Espera-se, ao final, ter uma engine testável, um roguelike mínimo jogável, leitura de dados em arquivos, scripts compilados, editor, exportação e pelo menos dois jogos de exemplo [Craddock 2015; Pinheiro and Oliveira 2020; Hendrikx et al. 2013; xUnit.net 2024; Martin 2017].

O próximo passo é a Etapa 1: montar o projeto, desenhar um mapa, mover o jogador e tratar colisão — provando na prática que *SadConsole é tela; Engine é regra* [Beck et al. 2001; Boehm 1988; Roguelike Celebration 2022; SadConsole 2024]. A ideia é contribuir com algo útil para quem quer fazer roguelike no Brasil, com código e documentação abertos [Pinheiro and Oliveira 2020; Hendrikx et al. 2011].

Referências

Ernest Adams and Joris Dormans. *Game Mechanics: Advanced Game Design*. New Riders, 2014.

Avalonia Team. Avalonia ui documentation. <https://docs.avaloniaui.net/>, 2024.

Kent Beck et al. Manifesto for agile software development. <https://agilemanifesto.org/>, 2001.

Berlin Interpretation. Definition of “roguelike”. RogueBasin Wiki, 2008. Acesso em 2026.

Jonas Bjorge and Johannes Torgersen. Entity component systems in practice. *Master’s Thesis, NTNU*, 2015.

Barry W. Boehm. A spiral model of software development and enhancement. *Computer*, 21(5):61–72, 1988.

Cameron Browne and Frederic Maire. Evolutionary game design. *IEEE Transactions on Computational Intelligence and AI in Games*, 2(1):1–16, 2010.

Mat Buckland. *Programming Game AI by Example*. Jones & Bartlett, 2005.

David L. Craddock. Building roguelikes with c# and monogame. Gamasutra / Game Developer, 2015.

Dungeon Crawl Stone Soup Team. Dungeon crawl stone soup: Design and development. <https://crawl.develz.org/>, 2023.

FireGiant. Wix toolset documentation. <https://www.firegiant.com/wixtoolset/>, 2024.

Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

Godot Engine. Godot engine documentation. <https://docs.godotengine.org/>, 2024.

Jason Gregory. *Game Engine Architecture*. CRC Press, 3rd edition, 2018.

Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. In *IEEE Transactions on Systems Science and Cybernetics*, volume 4, pages 100–107, 1968.

Mark Hendrikx, Sebastiaan Meijer, and Alexandru Iosup. Procedural content generation explained. *Anais do SBGames*, 2011.

Mark Hendrikx, Sebastiaan Meijer, Joost Van Der Velden, and Alexandru Iosup. Procedural content generation for games: A survey. *ACM Transactions on Multimedia Computing, Communications, and Applications*, 9(1):1–22, 2013.

JSON Schema. Json schema specification. <https://json-schema.org/>, 2024.

Raph Koster. A theory of fun for game design. Paraglyph Press, 2004.

Michael Lewis and Morgan McGuire. Game engine architecture for indie developers. *Journal of Game Design and Development Education*, 2014.

Robert C. Martin. *Clean Architecture*. Prentice Hall, 2017.

Microsoft Corporation. .net documentation. <https://learn.microsoft.com/dotnet/>, 2024.

Microsoft Learn. dotnet publish for .net. <https://learn.microsoft.com/dotnet/core/deploying/>, 2023.

Microsoft .NET Team. .net compiler platform (roslyn). <https://github.com/dotnet/roslyn>, 2024.

MonoGame Team. Monogame framework documentation. <https://docs.monogame.net/>, 2024.

Robert Nystrom. *Game Programming Patterns*. Genever Benning, 2014.

- Amit Patel. Introduction to the a* algorithm. Red Blob Games, 2014. <https://www.redblobgames.com/pathfinding/a-star/introduction.html>.
- Amit Patel. Field of view. Red Blob Games, 2016. <https://www.redblobgames.com/grids/visibility/>.
- Paulo R. F. Pinheiro and Marcos A. L. Oliveira. Desenvolvimento de jogos digitais com c# e .net. *Anais do SBGames*, 2020.
- Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. McGraw-Hill, 8th edition, 2014.
- Alexander Repenning, Andri Ioannidou, et al. Using human-centered design techniques to develop a visual language. *IEEE Symposium on Visual Languages and Human-Centric Computing*, 2017.
- RogueBasin. Dijkstra maps. RogueBasin Wiki, 2010.
- Roguelike Celebration. Roguelike celebration talks archive. <https://roguelike.club/>, 2022.
- SadConsole. Sadconsole documentation. <https://sadconsole.com/>, 2024.
- Noor Shaker, Julian Togelius, Mark J. Nelson, et al. *Procedural Content Generation in Games: A Textbook and Overview of Current Research*. Springer, 2016.
- Ian Sommerville. *Software Engineering*. Pearson, 10th edition, 2015.
- Julian Togelius, Georgios N. Yannakakis, Kenneth O. Stanley, and Cameron Browne. Search-based procedural content generation: A taxonomy and survey. In *IEEE Transactions on Computational Intelligence and AI in Games*, volume 3, pages 172–186, 2011.
- Julian Togelius, Noor Shaker, Mark J. Nelson, et al. *Procedural Content Generation in Games*. Springer, 2016.
- Paul Tozour. Building a near-optimal navigation mesh. *Game Programming Gems*, 2002.
- Unity Technologies. Unity engine documentation. <https://docs.unity3d.com/>, 2024.
- Tommy Wild. Roguelike development: Lessons from classic designs. Roguelike Celebration, 2018.
- xUnit.net. xunit.net documentation. <https://xunit.net/>, 2024.